

Trygve - Fagprøve Workshop

Meg :D

5. juni 2025

Innhold

1	Planlegging	2
1.1	Planleggings diagrammer	2
1.2	Avvik fra planen	5
2	Logg og Dokumentasjon	5
2.1	Database	5
2.2	Dag 1	5
2.3	Dag 2	5
2.4	Dag 3	6
2.5	Dag 4	6
3	Egenvurdering	8
3.1	Planlegging mot Gjennomføring	8
3.2	Utførelse og Kvalitet	8
3.2.1	C kode i C++ klær	9
3.2.2	Arkitektur	10
3.3	Faglige løsninger og utfordringer	10
3.3.1	PostGreSQL problemer	10
3.3.2	C++ utfordringer	10
3.3.3	Biblotek problemer	10
3.4	Samsvar mellom løsning og oppgave	10
3.5	Om jeg skulle fortsatt	11
3.6	Om jeg skulle gjentatt	11

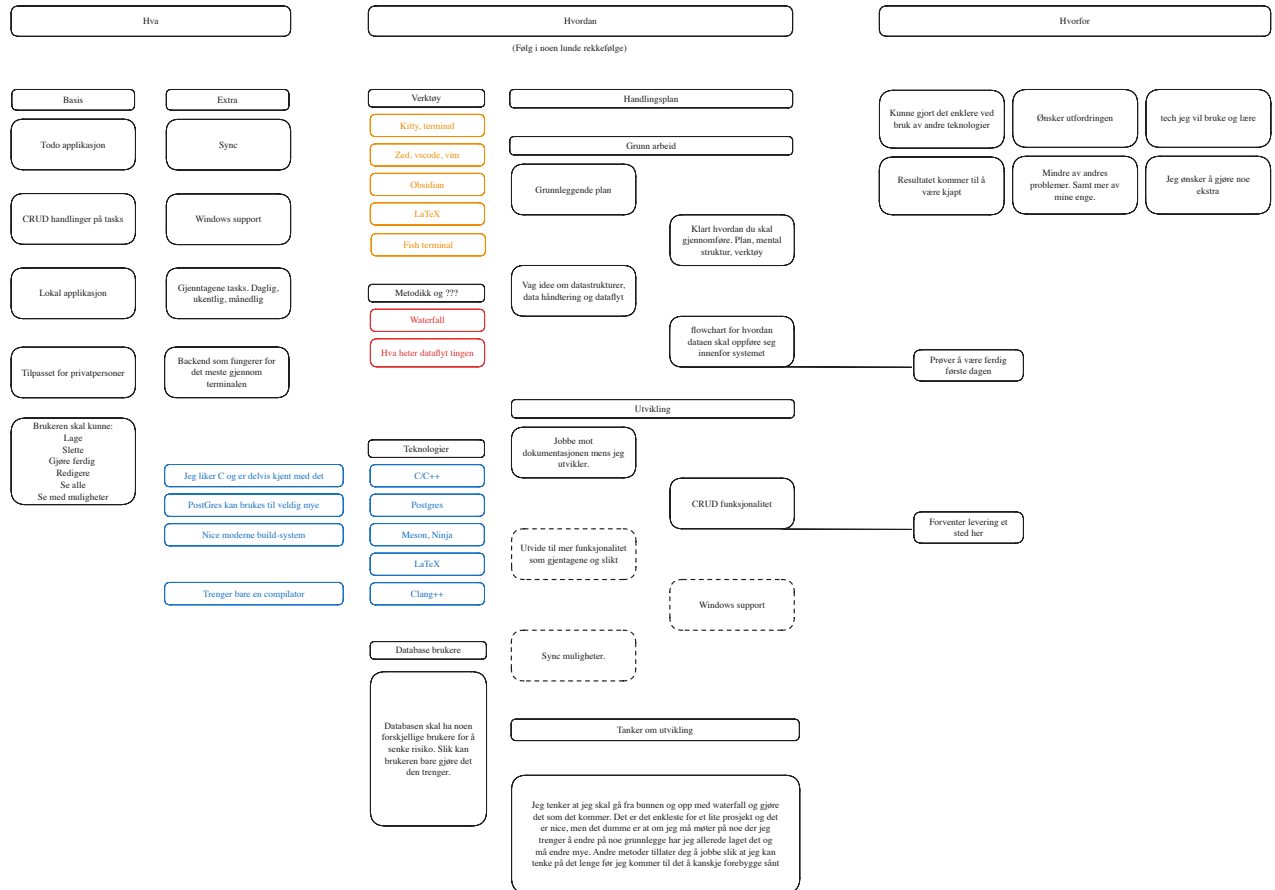
1 Planlegging

Planleggings delen tok sted første dagen og det var lite problemer utenom motivasjon denne dagen.

1.1 Planleggings diagrammer

Jeg ønsket å lage noe litt annerledes som passet min struktur bedre. Så jeg endte opp med et fly dokument heller enn en linear oppskrift. Synes ikke veldig mye om at alt på datamaskinen er som papir men digitalt. Se neste side for diagramene.

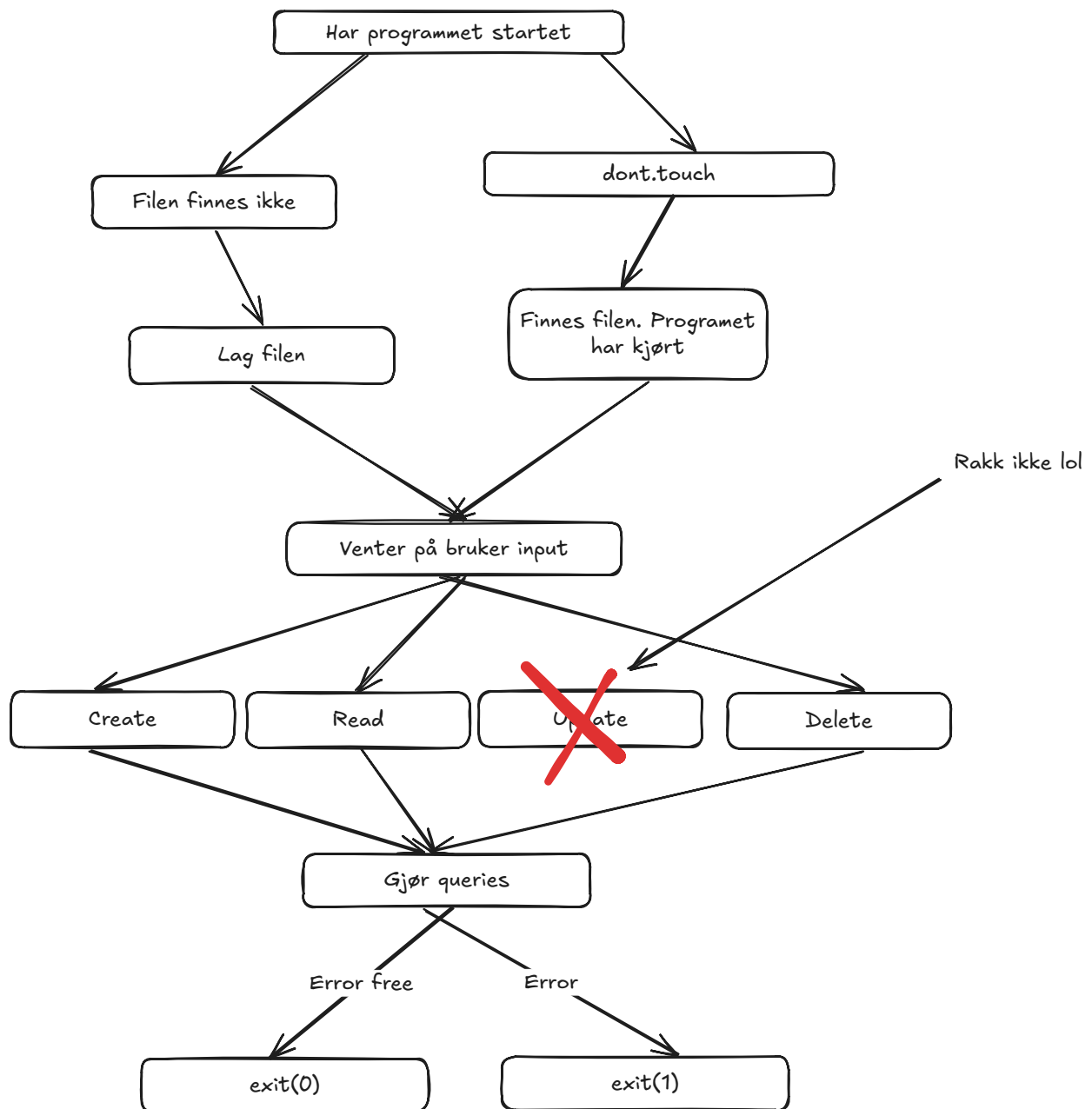
Overordnet



Flowchart



Flow av programmet



1.2 Avvik fra planen

Planen min var litt vag og prøvde å forutse avvik og gjøre det en del av oppgaven. Uansett var det et par underveis. Det som påvirker hele prosjekter er dette med at C++ koden min er ikke veldig C++, men minner mer på C. Jeg gjorde noen forsøk på å få det mer C++ var gjennom bruk av `unique_ptr`. Selv så brukte jeg ingen egendefinerte klasser.

2 Logg og Dokumentasjon

Jeg skrev de daglig loggene litt ujevnt. Fra den fjerde begynte de å bli bra.

2.1 Database

Eier: postgres (skal ikke brukes)

Datbase navn: WILLDO

Table navn: tasks

reader: wd_read_all_data placeholder

writer: wd_write_all_data placeholder

deleter: wd_delete_all_data placeholder

Database strukturer:

Field	Type
id	UUID
name	TEXT
description	TEXT
created_timestamp	DATE
completed_timestamp	DATE
repeating	BOOLEAN
completed	BOOLEAN

Tabell 1: Task Table Schema

2.2 Dag 1

Første dagen var brukt til planlegging. Jeg var mer enn bare litt rusten så det var litt slitsomt. Jobbet med en plan nesten hele dagen. Når jeg var ferdig hadde jeg en ganske solid plan. Bestemte meg også for å ikke bruke tiden på første dagen til å skrive logg som er hvorfor den er i fortids ting? Whatever

2.3 Dag 2

Dagen started litt tregt. Hadde noen problemer med postgres som ikke installerte, men så begynte det å gå litt raskere. Det har gått ish tre timer og jeg

har gjort en db connection lol. Tok meg tre sekunder bare setup som var tortur. Tror ikke jeg ligger veldig langt bak rn.

Jeg startet dagen litt senere enn jeg egentlig burde, men den startet bra. Nå først er det mye setup. Først må jeg lese dokumentasjonen til PostGres og finne ut av hvordan jeg kan gjøre en db connection og forsøke og sende over data.

Første jeg gjorde var å laste ned postgres så C++ bindingene for PostGres.

Hadde noen problemer med installationen og hadde noen problemer med å kompilere noen biblioteker selv så jeg bare installerte dem.

Nå er jeg klar til å begynne å programmere har gått en time ish og en del av grunnarbeidet som ikke er kode men heller sånn filstruktur og annet har en ganske god idee for det som krever for en mer generell plan som kan brukes til annet. Noe som

Grunnleggende: 1. Kunnskap

mappe og shit programmering arbeid whatnot

oja kanskje design idk alt som trengs før du begynner så selve jobben så det å bli ferdig.

2.4 Dag 3

Litt vanskelig å jobbe i dag. Kjenner at det ikke er sånn veldig stor utfordring som sliter på motivasjonen. Tenker jeg skal få CRUD operasjoner til å virke. Jeg har litt mindre anelse enn det jeg trodde på biblioteket jeg bruker hahaa. Kunne vært verre endringene de har gjort som gjør at GPT ikke skjønner det er veldig bra fra et sikkerhets og brukervennlighets punkt. Endringen fra å måtte escape all input til å bruke parameters er supert.

Sliter litt med noen brukere i PostGres veldig liten utfordring. Har endret en god del på ting, men jeg innså får en time ish siden at dette egentlig er C kode i klærne til C++. Kan skrive det om til å bruke klasser det er fullt mulig og noe jeg klarer, men da tror jeg ikke det blir noen fungerene prototype. Såååå jeg lager en liten demo bare vise at jeg kunne skrevet med klasser.

Ok det har gått noen timer jeg har skrevet store deler av funksjonene som skal gjøre CRUD tingene det gikk veldig som forventet. Det som tok lengst i seg selv var kanskje Postgres brukerne. orker ikke skrive så mye mer klokken er halv ni og kan sikkert fikse CLI tingene i morgen så man faktisk kan gjøre noe med programmet.

Melde på historie fortelling februar neste år

2.5 Dag 4

Litt mindre chil i dag. Har en del arbeid som må gjøres. Tenkte litt på det i går at postgres er veldig lite sende rundt bart. Det er litt mer sånn jeg er her for å bli database og burde ha brukt sqlite, men jeg har aldri brukt sqlite3 så det gjør jeg ikke nå. Kan ikke være så vanskelig, men noen ting jeg må ha funnet ut av som jeg ikke har tid til.

har ikke skrevet veldig mye specifict så quickfire round: Når jeg lager en `unique_ptr` så passer jeg en pointer til funksjonen eller jeg må men alle funksjo-

nene mine forventer en `pqxx::connection` men trenger en `pqxx::connection &`. Løsningen? Dereference til en reference. Ja idk men det virker så `*conn` altså data som pointes til put her til en reference `pqxx::connection &` som gjør at dataen er den samme og jeg ikke har to connections og den blir ikke kopiert. ikke veldig quick så langt.

```
1 std::unique_ptr<pqxx::connection> reader =
    establish_connection("wd_read_all_data", "placeholder");
```

I etterkant skulle jeg ønske å ikke ha tre wild men i en array ellerno kanskje ganske lett men jeg har ikke tid. Kan heller ikke ta CLI argumenter så det må gjøres tenker bare if statements. Så ja.

Wow man skulle ikke tro det var så vanskelig å skrive en loop men jeg skriver et navn for å lage noe og det blir veldig langt.

```
1 for (int x = 1; x < argc; ++x) {
2     std::string arg = argv[x];
3
4     if (arg == "list" && argc - 1 == x) {
5         list_tasks(*reader);
6         return 0;
7     }
8     else if (arg == "list" && x < argc - 1) {
9         list_task(*reader, argv[x + 1]);
10        return 0;
11    }
12    else if (arg == "create") {
13        std::string name, desc;
14
15        for (int i = x + 1; i < argc; ++i) {
16            std::string arg = argv[i];
17
18            if (arg == "-n") {
19                for (int y = i + 1; y < argc; ++y) {
20                    std::string n_arg = argv[y];
21
22                    if (n_arg == "-d") {
23                        if (y < argc) {
24                            i = y - 1; // jump forward for
                                next parse
25                        }
26                        // remove trailing space
27                        if (!name.empty()) {
28                            name.resize(name.length() - 1);
29                        }
30                        break;
31                    }
32
33                    name.append(n_arg);
34                    name.append(" ");
```

```

35         }
36     }
37 }
38 }
39 }

```

Literally det verste ever jeg hadde glemt å endre den innerste loopen fra `i` til `y` og for den andre description så sto det fortsatt `name.append()` enkelt å gjøre feil da

3 Egenvurdering

Dette er den lengste delen med stor margin.

3.1 Planlegging mot Gjennomføring

Planen min var ganske vag og beregnet på planer jeg hadde skrevet tidligere. Derfor ble jeg ganske rett på til der jeg kom. Viste hva jeg følte meg komfortabel med og hva jeg kunne gjennomføre. Noen ting kunne jeg ikke ha forholdt meg til i planen som skjedde uansett om jeg ville eller ikke. Dette var det med at koden min ikke var veldig C++ og at jeg var litt vel ivrig til å bruke postgres. Altså jeg skulle ønske at jeg hadde tid til å gå vekk i fra planen, men sånn ble det ikke.

3.2 Utførelse og Kvalitet

Generelt var kodekvaliteten min god og programmet var relativt modulært noe som er viktig om jeg ønsker å gjøre endringer i framtiden. Til slutt var jeg veldig fornøyd med CRUD funksjone, men ikke veldig fornøyd med return verdiene de gir. Eksempel

```

1 bool list_tasks(pqxx::connection &conn) {
2     std::string query{ "SELECT name FROM tasks" };
3     try {
4         //--snip--//
5     } catch (const pqxx::sql_error &e) {
6         //--snip--//
7     }
8     return false;
9 }

```

Som du ser så returnerer funksjonen en boolean verdi. Tanken her var å gjøre noe som ligner på C kode. Hvor 0 betyr ingen feil og noen annen verdi betyr at noe gikk galt. Nå pleier eldre C kode å returnne en `int` for å representere en verdi koblet til en `enum`. Jeg hadde ingen planer til det å tenkte jeg kunne komme billig unna ved å bare ha: feil, ikke feil.

Dette var før jeg var helt klar over hvordan `libpqxx` som var bindingen mellom PostgreSQL og C++. Som du kan se i eksemplet over bruker den `try catch`.

Så strukturen jeg brukte til feilkoder i funksjoner kunne vært gjort bedre om jeg spilte mer på dette.

I tillegg trenger jeg ikke `try`, `catch` block i hver funksjon. Noe jeg gjorde fordi ChatGPT løy til meg og jeg trodde på det fordi jeg ikke har så mye erfaring med det. Måten jeg gjorde det på var alright siden jeg kunne skrive spesifikke feilmeldinger for alle funksjonene. Fortsatt ikke optimalt ettersom det tar mye resurser. Om jeg skulle gjøre meg uten en `try`, `catch` i hver funksjon ville jeg ha bygget et mer avansert system i `main` funksjonen.

Rammeverket var veldig pent or ryddig. Føltes nesten ikke som et C++ bibliotek. Ingenting esoterisk trollmankunnskap nødvendig.

Minst ryddig er nok koden min som håndterer CLI argumentene. Den er modelert litt etter git sitt som jeg har sett på for å lære. Samtidig så tror jeg ikke at det er noen særlig ryddig måte å gjøre CLI parsing på.

3.2.1 C kode i C++ klær

Ikke et problem på samme måte som resten, men noe som plager meg er at koden min er nesten bare C kode og ikke veldig C++. Om jeg hadde vært litt bedre med tiden min kunne jeg ha rukket å faktorisere koden til å bruke en `DBConnector` klasse. Så tanken hadde da noe som dette:

```
1 export namespace Demo {
2     class Eve {
3     public:
4         void create_task(pqxx::connection &conn);
5         Eve(const std::string &conn_uri) {
6             reader(std::make_unique<pqxx::connection>(
7                 conn_uri));
8             writer(std::make_unique<pqxx::connection>(
9                 conn_uri));
10            deleter(std::make_unique<pqxx::connection>(
11                conn_uri));
12        };
13    private:
14        std::unique_ptr<pqxx::connection> reader;
15        std::unique_ptr<pqxx::connection> writer;
16        std::unique_ptr<pqxx::connection> deleter;
17    }
18 }
```

Det her er bare et eksempel på å vise at jeg kan skrive grunnleggende klasser. Jeg ønsker at disse skulle være i et array. Noe som dette:

```
1 enum ConnectionListing {
2     Reader 0,
3     Writer 1,
4     Deleter 2
5 }
```

```

6 std::unique_ptr<pqxx::connection>[] ConnectionCollection = {
    reader, writer, deleter};
7
8 // Accessing, something like this atleast
9 ConnectionCollection[ConnectionListing::Reader];

```

Trenger litt mer arbeid, men dette er bare vagt eksempel for å demonstrere.

3.2.2 Arkitektur

Programmet mitt er ikke optimalt designet for en GUI siden det ikke har noen loop, men dette er ikke et veldig stort problem. Hva man kan gjøre er å lage en GUI applikasjon som ikke har min app integert, men heller gjør CLI calls i bakgrunnen. Litt hacky, men veldig modulert.

3.3 Faglige løsninger og utfordringer

Det var ikke utrolig mye problemer.

3.3.1 PostgreSQL problemer

PostGres ga meg litt shit. Jeg skulle gjerne ha brukt SQLite3 siden jeg kan ha det på brukeren sin datamaskin, men det er ikke tilgjengelig for ARM64. Noen utfordringer med brukere hvor de må ha lov til å logge inn og gjøre ting i databasen. Var liksom to min. UUID er noe jeg har brukt en del lenge fordi det er unikt og whatever. Nå hadde det vært bedre med noe som bare teller oppover fordi da kunne brukere identifisere tasks med samme navn. Installere var heller ikke sånn veldig lett, men det var fordi jeg allerede hadde det på datamaskinen min så det var ikke veldig rart ting ikke virket.

3.3.2 C++ utfordringer

Argument parseren var litt vanskelig ettersom jeg har nested loops som alltid kommer til å gi meg noe shit. Jeg spurte ChatGPT om en løsning og han ga meg noe som ikke virket så da måtte jeg skrive det selv. Var en del skrive feil siden koden var nesten det samme så kopierte jeg over og tenkte ikke på det la kanskje til en 11 min med ekstra ting.

3.3.3 Bibliotek problemer

Var ikke kjent med libpqxx, men det var et veldig bra skrevet bibliotek og var lett å bruke.

3.4 Samsvar mellom løsning og oppgave

Løsningen min svarer ganske godt på oppgaven. Hvertfall gjennom rammene jeg satte for meg selv. Da er jeg spot on.

3.5 Om jeg skulle fortsatt

Om jeg skulle fortsatt hadde jeg:

1. Refaktorisert koden min til C++ kode
2. Byttet database til noe som brukeren kan ha på sin maskin
3. Lagt til repeating tasks
4. Gjort at koden kjører i en loop.
5. bygget et grafiskt grensesnitt
6. Sky synkroniseringsmuligheter

3.6 Om jeg skulle gjentatt

Om jeg skulle ha beygynt oppgaven en gang til hadde jeg prøvd å starte utvikling dag en. Utfordringer som hadde vært tydelige, men vanskelige å tenkte seg fram til hadde blitt enklere å forbrede seg for. Hatt brukerens behov litt mer i tankene og Bedt til himmelen om litt mer tid til utvikling så jeg ikke trengte å bare ha noe halveis. For en ting er det at jeg kan ikke helt ha halvferdige ting og samtidig utvikle videre ettersom det ikke hadde kompilert eller skapt problemer som man kan akseptere i Ecma, men kompilerte språk bare er litt strengere. Eller bare brukt versjons kontroll ellerno.